

this pointer

The **this** pointer is a pointer accessible only within the non-static member functions of a **class**, **struct**, or **union** type. It points to the object for which the member function is called. Static member functions don't have a **this** pointer.

Syntax

this

this->member-identifier

An object's **this** pointer isn't part of the object itself.

When a nonstatic member function is called for an object, the compiler passes the object's address to the function as a hidden argument.

For example, the following function call:

`myDate.setMonth( 3 );` can be interpreted as: `setMonth( &myDate, 3 );`

The object's address is available from within the member function as the **this** pointer. Most **this** pointer uses are implicit. It's legal, though unnecessary, to use an explicit **this** when referring to members of the class. For example:

```
void Date::setMonth( int mn )  
{  
    month = mn;          // These three statements  
    this->month = mn;     // are equivalent  
    (*this).month = mn;  
}
```

The expression `*this` is commonly used to return the current object from a member function:

```
return *this;
```

```
#include <iostream>
using namespace std;
class Demo {
private:
    int num;
    char ch;
public:
    void setMyValues(int num, char ch){
        this->num =num;
        this->ch=ch;
    }
    void displayMyValues(){
        cout<<num<<endl;
        cout<<ch;
    }
};
int main(){
    Demo obj;
    obj.setMyValues(100, 'A');
    obj.displayMyValues();
    return 0;
}
```

Pointers to classes using new operator

Objects can also be pointed to by pointers: Once declared, a class becomes a valid type, so it can be used as the type pointed to by a pointer. For example:

```
Rectangle * prect;
```

is a pointer to an object of class Rectangle.

Similarly as with plain data structures, the members of an object can be accessed directly from a pointer by using the arrow operator (->). Here is an example with some possible combinations:

```
// Example program
#include <iostream>
#include <string>
using namespace std;

int main()
{
    int *ptr = NULL;
    ptr = new int();
    int *var = new int(12);

    if(!ptr)
    {
        cout<<"bad memory allocation"<<endl;
    }
    else
    {
        cout<<"memory allocated successfully"<<endl;
        *ptr = 10;
        cout<<"*ptr = "<<*ptr<<endl;
        cout<<"*var = "<<*var<<endl;
    }
}

double *myarray = NULL;
myarray = new double[10];
if(!myarray)
{cout<<"memory not allocated"<<endl;}
else
{
    for(int i=0;i<10;i++)
        myarray[i] = i+1;
    cout<<"myarray values : ";
    for(int i=0;i<10;i++)
        cout<<myarray[i]<<"\t";
}
delete ptr;
delete var;
delete[] myarray;
return 0;
```

```
// pointer to classes example
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Rectangle {
```

```
    int width, height;
```

```
public:
```

```
    Rectangle(int x, int y) : width(x), height(y) {}
```

```
    int area(void) { return width * height; }
```

```
};
```

```
int main() {
```

```
    Rectangle obj (3, 4);
```

```
    Rectangle * foo, * bar, * baz;
```

```
    foo = &obj;
```

```
    bar = new Rectangle (5, 6);
```

```
    baz = new Rectangle[2] { {2,5}, {3,6} };
```

```
    cout << "obj's area: " << obj.area() << '\n';
```

```
    cout << "**foo's area: " << foo->area() << '\n';
```

```
    cout << "**bar's area: " << bar->area() << '\n';
```

```
    cout << "baz[0]'s area:" << baz[0].area() << '\n';
```

```
    cout << "baz[1]'s area:" << baz[1].area() << '\n';
```

```
    delete bar;
```

```
    delete[] baz;
```

```
    return 0;
```

```
}
```